

Báo cáo chính — HW06: Kiểm thử API

Sinh viên	23127195
Bài tập	HW06 — API Testing (HW06-AI)
SUT	EShop — ttbhanh/eshop-sut
Ngày thực hiện	2026-09-01
Công cụ AI	Claude Opus 5 (<code>claude-opus-5</code>) — khai báo đầy đủ tại ai/AI_AUDIT_REPORT.md
Công cụ kiểm thử	Postman 12.26.1 · Newman 6.2.2 · <code>newman-reporter-htmlextra</code> · GitHub Actions

Tóm tắt kết quả

Chỉ số	Giá trị
API kiểm thử	3 (mỗi API thuộc một Pool khác nhau)
Test case do AI sinh	110
Test case do sinh viên tự thêm	34
Tổng test case	144
Đã thi hành	144 (241 request, 746 assertion)
PASS	92
FAIL	52
Lỗi thật tìm được	24 (4 Critical · 7 High · 8 Medium · 5 Low)
Lỗi chỉ tìm được nhờ test case do người viết	6 / 24

1. Lựa chọn API

Chi tiết và lý do: [01_API_SELECTION.md](#) .

	Po ol	FR	Endpoint	Kỹ thuật trọng tâm
API -1	A	FR-04 — Quản lý hồ sơ cá nhân	<code>GET/PUT /api/users/me</code>	Phân vùng miền trên 3 tham số · SEC-06 leo quyền · SEC-01 rò rỉ dữ liệu
API -2	B	FR-09 — Áp dụng mã giảm giá	<code>POST /api/apply-coupon</code> <code>POST /api/coupon-usage</code>	Bảng quyết định 5 điều kiện C1–C5 · phân tích giá trị biên · oracle số học
API -3	C	FR-16 — Import sản phẩm từ CSV	<code>POST /api/admin/import-products</code>	Tính nguyên tử của giao dịch · phân quyền SEC-03 · phân vùng mảng × 5 trường

Thành viên khác trong nhóm đã nhận FR-02, FR-03, FR-08, FR-10, FR-14, FR-15 — bộ trên không trùng.

Phát hiện quan trọng khi phân tích đặc tả

api_specification.md không hề chứa SEC-01...SEC-07; các yêu cầu bảo mật nằm ở README.md (bản SRS) của SUT. Nếu chỉ đưa api_specification.md cho AI — đúng như câu chữ của đề bài ("Provide the SUT's API specification to an AI tool") — thì AI không thể sinh được test case bảo mật đúng ngữ cảnh. Prompt ở bước sinh vì vậy nạp cả hai tài liệu.

Ba trong số các lỗi nghiêm trọng nhất (BUG-A1-01, BUG-A2-03, BUG-A3-02) nằm đúng vào những chỗ mà đặc tả API im lặng còn SRS thì nói rõ.

2. Quy trình 5 bước cho từng API

Bước 1 — Sinh bằng AI

AI được dẫn qua sáu bước tách bạch, không dùng một prompt tổng quát. Toàn bộ prompt ở ai/prompts/PROMPT_LIBRARY.md :

Prompt	Nội dung	Ràng buộc âm quan trọng nhất
P1	Trích xuất đặc tả thành EndpointModel	Chưa được sinh test case; không được xem response thật
P2	Phân vùng miền theo danh mục quy tắc	Không được tự nghĩ test case theo cảm hứng
P3	Chuỗi chuyển trạng thái	Không được coi message trả về là bằng chứng
P4	Test case bảo mật	Không được dừng ở "API có trả 403 không"
P5	Kiểm tra lược đồ	Bắt buộc kiểm cả chiều "không có trường thừa"
P6	Tự phê bình bộ test vừa sinh	—

Điểm mấu chốt: tính đầy đủ đến từ danh mục quy tắc phân vùng, không đến từ mô hình. Với mỗi tham số, danh mục bắt buộc phủ: thiếu trường · null · sai kiểu · chuỗi rỗng · chỉ khoảng trắng · Unicode · sáu điểm biên min-1/min/min+1/max-1/max/max+1 · khớp mẫu / không khớp / khớp một phần · khoá ngoại tồn tại / không tồn tại / 0 / âm.

Nếu chỉ hỏi "sinh test phân vùng", AI bỏ qua điểm v == min — đúng điểm tìm ra BUG-A2-03.

Ràng buộc quan trọng nhất: ở bước sinh, AI không được xem response thật của hệ thống. Nếu xem, kỳ vọng sẽ bị neo theo hành vi hiện có và mọi lỗi sẽ được hợp thức hoá thành "đúng thiết kế".

Bước 2 — Audit (rà soát của con người)

Mỗi test case do AI sinh được gắn nhãn VALID / INVALID / INCOMPLETE kèm lý do, ghi thẳng trong nguồn test case và xuất ra cột Nhãn audit của file Excel.

API	VALID	INCOMPLETE (đã hiệu chỉnh)	INVALID
API-1	31	4	0
API-2	36	3	0
API-3	34	2	0
Tổng	101	9	0

9 test case bị hiệu chỉnh — tất cả đều cùng một dạng sai: AI ràng buộc `400` cho những đầu vào "trông có vẻ sai" kể cả khi đặc tả **không hề quy định**. Ví dụ:

Test case	AI kỳ vọng	Hiệu chỉnh của SV	Lý do
TC-A1-005	400 cho tên dài 500 ký tự	<code>statusIn [200,400,413]</code>	SRS không đặt giới hạn độ dài cho họ tên (chỉ FR-15 đặt 255 cho tên sản phẩm)
TC-A1-019	400 cho phone có khoảng trắng bao quanh	<code>statusIn [200,400] +</code> kiểm giá trị lưu	SRS không quy định có trim hay không
TC-A1-025	400 cho <code>shipping_address = null</code>	200	Đây là trường không bắt buộc ; null là cách hợp lệ để người dùng xoá địa chỉ
TC-A2-045	400 cho <code>total_amount</code> thập phân	"không được 5xx"	SRS không cấm số thực
TC-A3-029	"báo lỗi" cho <code>category_id</code> dạng chuỗi	ép kiểu tương minh hoặc từ chối	Dữ liệu từ CSV luôn là chuỗi — đây là trường hợp mặc định, không phải ngoại lệ

Ngoài ra 4 test case bị AI **phân loại sai kỹ thuật** (xếp kiểm thử hiệu năng vào nhóm schema) — được giữ lại nhưng ghi rõ đây là kiểm thử phi chức năng với ngưỡng tự đặt, không có trong SRS.

Bước 3 — Mở rộng (34 test case do sinh viên tự thêm)

Sáu câu hỏi dẫn đường, và lỗi mà mỗi câu tìm ra:

#	Câu hỏi	Test case tiêu biểu	Lỗi tìm được	Vì sao AI bỏ sót
1	Nếu client chỉ gửi một phần dữ liệu thì sao?	TC-A1-028	BUG-A1-05 mất dữ liệu âm thầm	Đặc tả liệt kê 3 trường; không chỗ nào nói " <i>client có thể chỉ gửi một trường</i> ". AI sinh request theo đúng ví dụ trong tài liệu.
2	Vị trí của phần tử lỗi có làm đổi kết luận không?	TC-A3-033 , TC-A3-034	BUG-A3-02 (kết luận đúng về tính nguyên tử)	"Chạy tuần tự rồi dừng" và "giao dịch nguyên tử" cho cùng kết quả khi lỗi ở dòng đầu. Chỉ test một vị trí sẽ kết luận sai.
3	Báo cáo có nhất quán với trạng thái thật của CSDL không?	TC-A3-036	BUG-A3-02 (mức nghiêm trọng thật)	AI kiểm "có báo lỗi không" và "CSDL có đổi không" như hai việc rời rạc, không kiểm mâu thuẫn giữa chúng.
4	Có bất biến nghiệp vụ nào bắt được mọi cách tính sai?	TC-A2-036 , TC-A2-037	BUG-A2-02	AI viết oracle theo từng giá trị cụ thể. Bất biến $0 \leq \text{giảm giá} \leq \text{tổng đơn}$ bắt được cả những cách tính sai chưa nghĩ ra.
5	Hậu quả nghiệp vụ thật sự là gì?	TC-A1-038 , TC-A3-041	BUG-A1-01, BUG-A3-01 (nâng từ Medium lên Critical)	AI dừng ở "API có trả 403 không". Đi thêm một bước mới chứng minh được chiếm quyền hệ thống / hàng giả lên sàn.
6	Ngữ nghĩa ngôn ngữ cài đặt có tạo khe hở?	TC-A3-013 , TC-A2-043	BUG-A3-06	Cần biết " " là truthy trong JavaScript, và "90000" > "300000" là sai khi so sánh chuỗi. Kiến thức về cài đặt, không có trong đặc tả.

Ba test case bảo mật khác cũng do người thêm: TC-A1-035 (JWT ký sai khoá), TC-A1-036 (alg=none), TC-A3-042 — đều đòi hiểu biết về cách hiện thực JWT chứ không suy ra được từ đặc tả.

- Nguyên nhân gốc AI bỏ sót**, gom lại thành ba nhóm:
- 1. **Chất lượng prompt** — prompt hướng AI bám sát tài liệu, nên nó không thử những thứ *ngoài* tài liệu (câu 1, 6).
 - 2. **Giới hạn mô hình** — AI kiểm tra từng thuộc tính riêng lẻ, không tự đặt câu hỏi về *quan hệ* giữa các quan sát (câu 3, 4).
 - 3. **Đặc thù API** — hậu quả nghiệp vụ chỉ hiện ra khi nối nhiều endpoint lại (câu 2, 5).

Bước 4 — Thi hành

```
bash scripts/run_newman.sh          # restart SUT rồi chạy cả 3 collection
```

Mọi request mang header X-Student-Id: 23127195 , gắn bằng pre-request script cấp collection kèm console.log làm bằng chứng theo \$11:

```
pm.request.headers.upsert({ key: 'X-Student-Id', value: sid });
console.log('[X-Student-Id] ' + sid + ' -> ' + pm.request.method + ' ' + pm.request.url.getPath());
```

Collection	Request	Assertion	FAIL	Test case FAIL
api1_fr04_user_profile	88	256	23	20
api2_fr09_apply_coupon	57	210	20	12
api3_fr16_import_products	96	280	30	20
Tổng	241	746	73	52

Báo cáo HTML: [newman/](#) . Danh sách tính năng Postman đã dùng: [02_POSTMAN_FEATURES.md](#) .

⚠ Hai khiếm khuyết của **chính bộ test** phát hiện ở bước này

Đây là phần quan trọng nhất của báo cáo — và là bằng chứng cho thấy bước rà soát của con người không thể bỏ.

(a) Assertion bất đồng bộ nuốt mất lỗi. AI sinh assertion theo mẫu gọi `pm.sendRequest` rồi đặt `done()` ngay sau `pm.expect` . Khi assertion **đạt** → PASS bình thường. Khi assertion **hỏng** → ngoại lệ ném trước `done()` → Newman **âm thầm bỏ qua** test đó: không PASS, không FAIL, biến mất khỏi báo cáo.

Hậu quả: TC-A1-037 — test cho **BUG-A1-01**, lỗi nghiêm trọng nhất của cả bài — ban đầu hiện ra như "đã pass". Chỉ lộ ra khi đối chiếu báo cáo Newman với kết quả `curl` thủ công: `curl` cho thấy `role = admin` trong khi Newman không có dòng FAIL nào tương ứng. **Dấu hiệu là sự vắng mặt của một assertion, không phải một assertion sai.**

Đã sửa: bọc `try { ... done(); } catch (e) { done(e); }` cho **32 assertion** — sửa một chỗ trong bộ biên dịch, áp cho toàn bộ.

(b) Dữ liệu chuẩn bị trong pre-request script không đáng tin. Chụp mốc `countBefore` bằng `pm.sendRequest` trong pre-request script không đảm bảo hoàn tất trước khi request chính được gửi → 3 kết quả **FAIL giả**. Đã thay bằng **26 request [SETUP] tường minh**.

Sau khi sửa, 3 FAIL giả biến mất và **3 lỗi thật mới lộ ra** (TC-A1-028 , TC-A1-037 , TC-A3-032/033/034).

Bước 5 — Báo cáo lỗi

Chi tiết đầy đủ: [bugs/BUG_REPORTS.md](#) .

Tải hiện đọc lập bằng `curl` : `bash bugs/reproduce_bugs.sh` → [bugs/evidence/reproduce_output.txt](#) .

Cả **24 lỗi đã được tạo thành GitHub Issue** trên repo bài tập: [issue #5](#) → [#28](#) (nhấn `hw06-23127195`). Nội dung gốc: [bugs/GITHUB_ISSUES.md](#) .

Bốn lỗi Critical

Mã	Vi phạm	Mô tả ngắn
BUG-A1-01	SEC-06	PUT /api/users/me nhận trường role từ client → bất kỳ người dùng nào cũng tự nâng lên admin bằng một request. Sau đó mở được toàn bộ /api/admin/*.
BUG-A2-01	FR-09 C4, SEC-02	POST /api/apply-coupon không yêu cầu đăng nhập — một trong năm điều kiện bắt buộc của FR-09 hoàn toàn không được cài đặt.
BUG-A2-02	FR-09	Công thức percent tính total × (1 - value) thay vì total × value / 100 → giảm giá âm: khách dùng mã giảm 10% phải trả gấp 10 lần giá gốc.
BUG-A3-01	SEC-03, FR-12	Người dùng thường import được sản phẩm lên cửa hàng — kèm tên, giá, mô tả và imageUrl do họ kiểm soát. Kết hợp BUG-A3-11 thành vector stored XSS.

Phân bố theo mức độ

Mức	Số lượng	Mã
 Critical	4	A1-01, A2-01, A2-02, A3-01
 High	7	A1-02, A2-03, A2-04, A2-05, A3-02, A3-03, A3-09
 Medium	8	A1-03, A1-04, A1-05, A2-08, A3-04, A3-05, A3-08, A3-11
 Low	5	A2-06, A2-07, A3-06, A3-07, A3-10

3. Tích hợp CI/CD

Chi tiết: [ci/CI_CD_REPORT.md](#) · Workflow: [.github/workflows/newman-23127195.yml](#)

Vấn đề thiết kế: SUT có 24 lỗi nên chạy cả 144 test case trong CI thì pipeline **luôn đỏ** — và một pipeline luôn đỏ sẽ bị bỏ qua sau vài ngày, mất hết giá trị cảnh báo.

Giải pháp — hai tầng:

Tầng	Nội dung	Chặn build?	Kết quả
1 — Baseline	92 test case đang đạt	 Có	550 assertion, 0 FAIL
2 — Full suite	144 test case	 Không	52 FAIL (kết quả mong đợi)
2b — Data-driven	39 iteration từ 3 file CSV	 Không	—

Khi một lỗi được sửa, test case tương ứng được thêm vào [ci/baseline_allowlist.json](#) và trở thành một phần của cổng chặn — bảo vệ bản sửa đó khỏi bị phá vỡ lần sau.

Pipeline còn có một bước **chống lệch nguồn**: chạy lại các script sinh collection rồi `git diff`; nếu collection trong repo không khớp với `testcases/*.json` thì build đỏ ngay.

4. Agent Skill — bộ sinh test case API bằng AI

Thiết kế đầy đủ: [agent-skill/DESIGN.md](#) ·

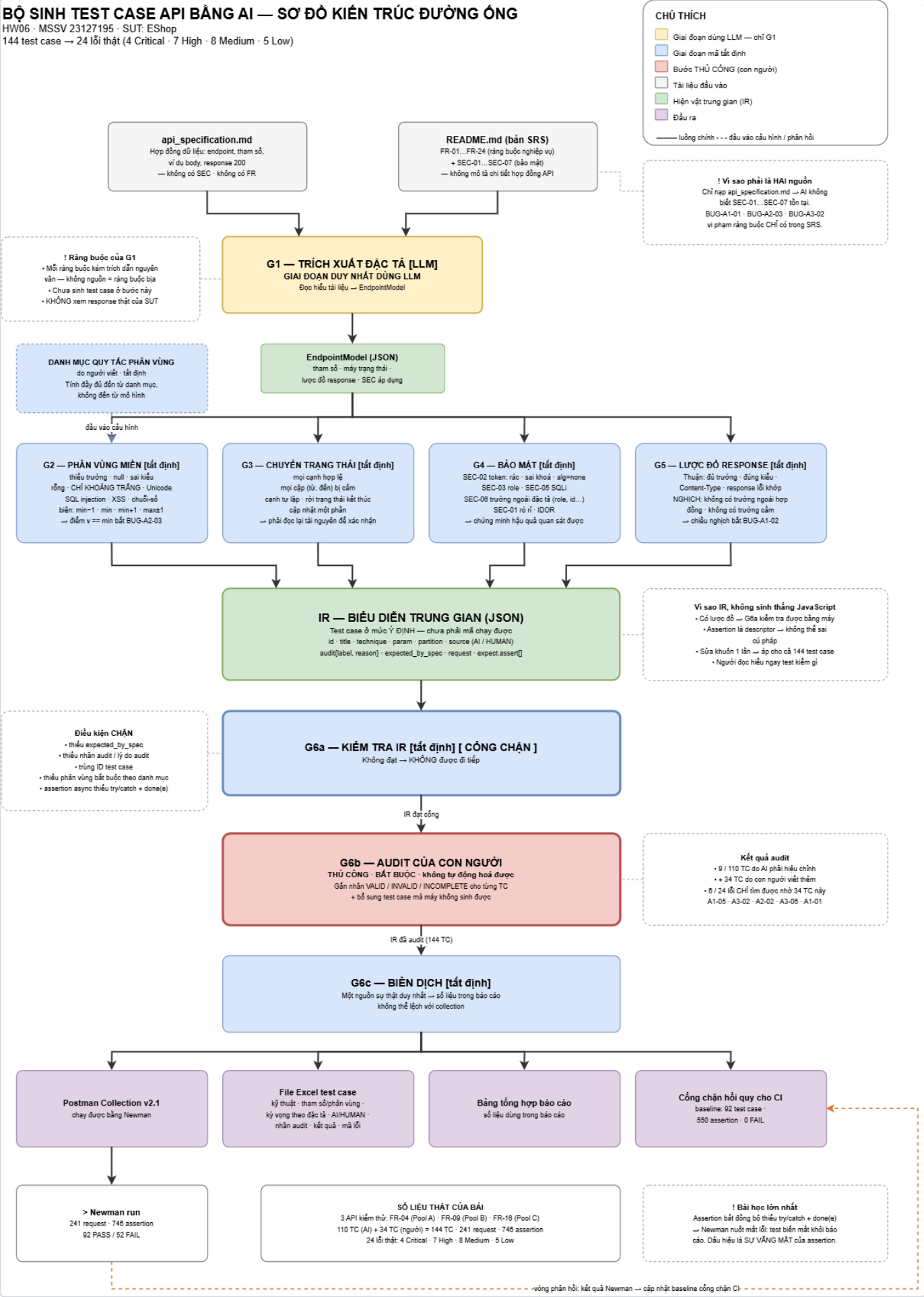
Pseudocode: [agent-skill/pseudocode/generator_pseudocode.md](#) ·

Cài đặt tham chiếu: [agent-skill/pseudocode/generator.py](#)

Ý tưởng trung tâm: tách phần suy luận ngôn ngữ ra khỏi phần sinh mã. Trong sáu giai đoạn G1–G6, **chỉ G1 dùng LLM** (trích xuất đặc tả thành `EndpointModel`); năm giai đoạn còn lại là mã tắt định. Tính đầy đủ đến từ danh mục quy tắc, không từ mô hình.

BỘ SINH TEST CASE API BẰNG AI — SƠ ĐỒ KIẾN TRÚC ĐƯỜNG ỒNG

HW06 · MSSV 23127195 · SUT: EShop
144 test case → 24 lỗi thật (4 Critical · 7 High · 8 Medium · 5 Low)



Sơ đồ do sinh viên 23127195 tự vẽ bằng draw.io theo yêu cầu §11. File nguồn:

[agent-skill/diagram/ai_test_generator_diagram.drawio](#)

— mở lại được để kiểm chứng quá trình dựng hình.

Cách đọc sơ đồ. Màu phân theo *trách nhiệm*, không theo thứ tự: ■ vàng = giai đoạn dùng LLM (duy nhất G1) · ■ xanh = mã tắt định · ■ hồng = bước thủ công bắt buộc của con người (G6b) · ■ xanh lá = hiện vật trung gian (EndpointModel , IR) · ■ tím = đầu ra. Mũi tên đứt màu cam ở đây là vòng phản hồi: kết quả Newman cập nhật lại allowlist của cổng chặn CI.

Kiểm chứng thiết kế: `python agent-skill/pseudocode/generator.py --demo` sinh **44 test case** cho FR-04 từ một EndpointModel , qua bộ kiểm tra G6a với **0 lỗi** — rất gần con số 45 mà quy trình đầy đủ tạo ra, cho thấy thiết kế phản ánh đúng việc đã làm chứ không phải mô tả lý thuyết.

Bộ kiểm tra G6a nay **chặn** cả hai khiếm khuyết đã gặp thật ở §2 bước 4: assertion bất đồng bộ thiếu try/catch, và tác dụng phụ đặt trong pre-request script.

✓

Về §11 (Anti-AI-Cheat). Sơ đồ trên do sinh viên tự dựng bằng draw.io — bố cục, phân nhóm màu và cách sắp xếp là quyết định của sinh viên. Repo **không** chứa bản vẽ nào do AI sinh (kể cả Mermaid/ASCII); bản phác ASCII trước đây đã được gỡ bỏ để tránh nhập nhằng. Ghi chú quá trình: [agent-skill/diagram/README.md](#) .

5. Phần AI

Tài liệu	Nội dung
ai/AI_AUDIT_REPORT.md	Khai báo bắt buộc · bảng công cụ · 12 lượt tương tác · 4 nhóm sai sót của AI đã được sửa
ai/AI_CRITIQUE.md	Phê bình AI, 296 từ
ai/interactions/SESSION-01_2026-09-01.md	Nhật ký INT-01...INT-12: prompt nguyên văn, suy luận của AI, output thật, phán quyết review
ai/prompts/PROMPT_LIBRARY.md	6 prompt tương ứng 6 bước, kèm nhận xét hiệu quả từng prompt

Số liệu kiểm toán AI: 110 test case do AI sinh — 101 VALID , 9 INCOMPLETE đã hiệu chỉnh, 0 INVALID . Nhưng ở tầng hạ tầng, AI mắc **3 lỗi có hệ thống** (§2 bước 4 và §4.3 của báo cáo audit) — mỗi lỗi nhân lên hàng chục assertion. Đó là kết luận chính: *AI làm tốt ở mức từng test case, kém ở mức khuôn mã dùng chung, và không tự phát hiện được lỗi của chính nó.*

6. Cấu trúc thư mục

23127195/	
└ docs/	00_MAIN_REPORT.md (tài liệu này) · 01_API_SELECTION.md · 02_POSTMAN_FEATURES.md
└ testcases/	*_testcases.json (nguồn sự thật) · TESTCASES_23127195.xlsx · *.csv · TEST_SUMMARY.md
└ postman/	collections/ (9) · environments/ · data/ (3 CSV) · scripts/ (3 bộ sinh)
└ newman/	báo cáo HTML/JSON/JUnit/console của mọi lần chạy
└ bugs/	BUG_REPORTS.md · GITHUB_ISSUES.md · reproduce_bugs.sh · evidence/ · screenshots/
└ ci/	CI_CD_REPORT.md · baseline_allowlist.json · evidence/
└ agent-skill/	DESIGN.md · SKILL.md · pseudocode/ · diagram/
└ ai/	AI_AUDIT_REPORT.md · AI_CRITIQUE.md · interactions/ · prompts/
└ evidence/	EVIDENCE_INDEX.md · git_commit_log.txt
└ scripts/	run_newman.sh · export_testcases.py
└ video/	kịch bản và checklist quay demo

7. Những việc sinh viên phải tự hoàn tất

Theo §11 của đề bài, các hạng mục sau **không được** do AI tạo và TA sẽ kiểm tra khi chấm:

#	Việc	Trạng thái	Bằng chứng
1	Vẽ sơ đồ bộ sinh test case	✓ Xong	Tự vẽ bằng draw.io, kèm file nguồn <code>.drawio</code> mở lại được; nhúng ở §4
2	Chụp Postman Console ([X-Student-Id] 23127195 -> ...)	✓ Xong	3 ảnh trong <code>evidence/</code> ; ảnh <code>_timestamps</code> bung sẵn <i>Request Headers</i> nên thấy cả header thật trên đường truyền lẫn <code>Host: localhost:3000</code>
3	24 GitHub Issue kèm ảnh chụp	✓ Xong	<code>#5 → #28</code> ; 32 ảnh nhúng trong thân từng issue, mỗi ảnh hiện lệnh <code>curl</code> đầy đủ
4	CI chạy thật, 2 lần chạy mẫu	✓ Xong	<code>run #3</code> xanh · <code>run #4</code> đỏ đúng 1 test case, cùng trong PR #35
5	Video demo Agent Skill	✓ Xong	<code>https://youtu.be/KsxYU52I4WE</code> — "Demo api testing agent skill"
6	Xác nhận không trùng API với 23127060 và 23127259	✓ Xong	9 FR đôi một khác nhau — bảng đối chiếu ở <code>01_API_SELECTION.md</code> §5
7	Xuất PDF toàn bộ báo cáo	✓ Xong	10 file trong <code>pdf/</code>